



UNIVERSITY OF HERTFORDSHIRE
School of Physics, Engineering and Computer Science

MSc Cybersecurity
7COM1039-0206-2025 Advanced Computer Science Masters
Project
April 2026

**Machine Learning-Based Detection of Phishing Websites
Using URL and Domain Features**

Name: Justin Jamil
Student ID:24065217
Supervisor: Pusp Joshi

I confirm that I have critically proof-read and quality checked this report, ensuring it is free from grammar, spelling, and formatting errors, and that it meets the required standards of coherence and presentation.

Contents

2. Introduction.....	7
2.1 Problem Overview	7
2.2 Current Limitations and Research Gap.....	7
2.3 Project Aim, Research Question, and Objectives	8
2.4 Novelty and Contribution	8
2.5 Ethical and Commercial Context.....	9
3. Literature Review.....	10
3.1 Introduction to the Field.....	10
3.2 Blacklist and Heuristic Approaches: Reactive and Brittle.....	10
3.3 Machine Learning Approaches: Comparative Analysis	10
3.4 Feature Engineering: What Predicts Phishing?.....	11
3.5 Hybrid Architectures and Research Gaps	12
4. Methodology.....	13
4.1 Research and Development Framework: CRISP-DM	13
4.2 Dataset.....	13
4.3 Data Preparation.....	14
4.4 Model Selection and Justification.....	14
4.5 Feature Engineering: The 31-Feature Framework.....	14
4.6 System Architecture: PhishGuard.....	15
4.7 Hybrid Detection: Rule-Based Override Layer	15
4.8 Testing Strategy and Validation.....	15
4.9 Ethical and Legal Considerations	16
5. Results and Quality.....	17
5.1 Model Performance Comparison	17
5.2 Confusion Matrix Analysis	18
5.3 Feature Importance Analysis	19

5.4 Hybrid Override Impact.....	21
5.5 Technical Challenges and Solutions	22
5.5.1 Private IP Address False Positive	22
5.5.2 Ambiguous Confidence from Missing API Features.....	22
5.5.3 Live HTML Fetch Failures	22
5.6 End-to-End System Validation	22
6. Evaluation and Conclusion	24
6.1 Evaluation Against Objectives.....	24
6.2 Project Management Reflection and Comparison with Literature	24
6.3 Critical Evaluation of Limitations	25
6.4 Commercial and Academic Significance.....	26
6.5 Future Work and Conclusion.....	26
7. References.....	28
Appendices.....	30
Appendix A: Full 31-Feature Reference Table.....	30
Appendix B: Key Code Samples	32
B1: Private IP Detection Fix (check_dns function, app.py)	32
B2: Hybrid Override Layer (/predict route, app.py)	32
Appendix C: Feature Importance Chart — Random Forest	32
List of Figures	
Declaration	

List of Figures

Figure 1: Model performance comparison — Logistic Regression vs Random Forest across all four metrics.

Figure 2: Confusion matrix visualisation — Random Forest classifier (test set, $n = 2,211$).

Figure 3: Top 10 feature importances. Red bars = features unavailable at inference time.

Figure 4: Impact of the hybrid override on prediction confidence distribution.

Figure 5: Live validation results — correctly classified vs misclassified across both URL categories ($n = 60$).

Figure C1: Feature importance scores for all 31 Random Forest features, ranked by mean decrease in Gini impurity (Appendix C).

Note: Figures 1–5 and Figure C1 are embedded in their respective sections within the report body and appendices.

Declaration

MSc Final Project Declaration (without requiring ethics approval)

I, Justin Jamil (Student ID: 24065217), declare that this Final Project Report, submitted in partial fulfilment of the requirements for the degree of MSc Cybersecurity at the University of Hertfordshire, is my own original work. This project did not involve any third-party participants or require ethics committee approval.

I confirm that:

- The work presented in this report is my own, unless otherwise acknowledged through appropriate citation and referencing.
- I have not plagiarised any work from other sources, and all sources have been duly cited using the Harvard referencing system.
- GenAI tools were used only for proofreading purposes, not for generating any substantive content of this report.
- I understand the University's regulations on academic integrity and academic misconduct as set out in UPR AS14.

Signed: Justin Date: April 2026

Name: Justin Jamil Student ID: 24065217

Abstract

Over 1.3 million bogus websites were found in the first quarter of 2023 alone, making phishing attacks one of the most prevalent and economically harmful types of cybercrime (APWG, 2023). Through the examination of URL-based, HTML-based, and domain-level elements, this research explores the degree to which machine learning techniques can be used to identify phishing websites. Which machine learning model uses lexical and structural URL information to detect phishing websites most effectively is the main research issue addressed.

Two classification models were applied and assessed using the UCI Phishing Websites Dataset, which consists of 11,055 labelled website samples characterised by 31 features: Random Forest as the main model and Logistic Regression as a baseline. With an accuracy of 96.79%, precision of 96.94%, recall of 97.33%, and F1-score of 97.13%, the Random Forest classifier outperformed Logistic Regression in every statistic. The Random Forest's improved recall is especially important in the context of cybersecurity because it means that fewer phishing sites evade detection, hence lowering user susceptibility to identity theft. The ratio of suspicious anchor tags in page HTML, or AnchorURL, was found to be the single strongest predictor at about 23% importance in a feature importance analysis, demonstrating the necessity of live HTML analysis for accurate detection.

This project created PhishGuard, a Flask-based web application with real-time URL analysis employing 31 live-extracted features, in addition to the machine learning pipeline. By combining context-aware feature defaults with a rule-based override mechanism, a new hybrid detection layer makes up for the absence of five third-party API capabilities. During live testing, significant technical issues were found and fixed, such as private IP false positives and unclear confidence scoring. The results verify that Random Forest offers a reliable and reasonably implementable basis for phishing website classification when enhanced by hybrid detection logic. In order to recover the roughly 21% of model importance currently lost due to API unavailability, future work should concentrate retraining on modern datasets and interaction with free-tier threat intelligence APIs.

2. Introduction

2.1 Problem Overview

Phishing is a kind of social engineering assault in which hackers create emails or websites that appear to be from legitimate, well-known businesses in order to fool consumers into divulging financial information, login credentials, or personal information. Phishing is one of the most prevalent and harmful cybersecurity dangers, despite years of education programs and protection measures. Over 1.3 million distinct phishing websites have been identified in just the first quarter of 2023, according to the Anti-Phishing Working Group (APWG), indicating that the issue is still becoming worse (APWG, 2023).

Phishing has significant financial repercussions. Phishing is one of the most frequent beginning vectors for data breaches, with an average cost of USD 4.76 million per incident, according to the IBM Cost of a Data Breach Report (2023). Successful phishing campaigns often lead to ransomware deployment, credential theft, and regulatory penalties under frameworks like the UK General Data Protection Regulation (UK GDPR) at the organisational level. Smaller organisations without dedicated security teams are disproportionately affected by these outcomes. Individual victims experience long-term reputational damage, identity theft, and financial loss. The financial justification for investing in automated, proactive detection systems that can function more quickly and on a larger scale than human analysts is directly shaped by the scope and endurance of phishing as a threat vector. These effects highlight how urgent it is to have more dependable, automated methods for identifying phishing websites that can function proactively as opposed to reactively.

2.2 Current Limitations and Research Gap

Blacklist-based systems, heuristic approaches, and machine learning-based classifiers are the three main types of phishing detection techniques now in use. Although curated databases of known dangerous URLs are the foundation of blacklist systems like Google Safe Browsing, these systems are reactive by nature and are unable to detect zero-day phishing sites that have not yet been reported (Oest et al., 2020). Phishing pages accomplish most of their credential harvesting during the first hour of activation, before most blacklists are updated, according to research by Oest et al. (2020). This makes this class of solution fundamentally inadequate against contemporary, transient operations. Faster response times are provided by heuristic rule-based methods, but they also need ongoing manual maintenance and often result in higher false-positive rates when attackers modify their evasion strategies (Mohammad et al., 2014).

By allowing systems to learn discriminative patterns from labelled data without the need for explicit rule engineering, machine learning presents a viable proactive alternative. However, in most published investigations, there are still two important gaps. First, the significant engineering difficulty of calculating equivalent features from raw URL inputs at deployment time is not addressed by almost all research, which assess models on pre-extracted, static dataset characteristics. Second, there is a systematic blind hole that is recognised but seldom fixed since a subset of high-signal characteristics, such as domain traffic rank, Google indexing status, and

page rank, rely on third-party APIs that are either outdated or economically restricted. Instead of being a benchmarking exercise, this project was intended to directly solve both deficiencies and create a functional solution.

2.3 Project Aim, Research Question, and Objectives

The aim of this project is to investigate how effectively machine learning models can detect phishing websites by analysing URL-based and domain-based features, and to determine the most reliable model for practical deployment. The central research question is:

“Which machine learning model provides the most effective phishing website detection using lexical and structural URL features?”

To address this question, the following six objectives were defined:

1. To conduct a critical review of existing phishing detection methods, with a focus on machine learning approaches and feature engineering strategies.
2. To implement and evaluate Logistic Regression and Random Forest classifiers using the UCI Phishing Websites Dataset.
3. To perform feature importance analysis to identify the most discriminative predictors of phishing behaviour.
4. To develop a real-time URL feature extraction module capable of computing all 31 features from a raw URL input.
5. To design and deploy a functional web-based prototype (PhishGuard) integrating the trained model with a hybrid rule-based override layer.
6. To critically evaluate the system’s performance, identify limitations, and propose evidence-based directions for future improvement.

2.4 Novelty and Contribution

The creation of a hybrid detection architecture that combines a trained Random Forest classifier with a rule-based override layer and context-aware feature defaults is the project's main contribution. The lack of third-party API data is specifically compensated for in this design; this is a disadvantage that almost all previous deployment-oriented research recognise but none address with a specified technical approach. Although two-stage designs were suggested by research like Rao and Pais (2018), neither the particular issue of API unavailability nor the confidence bias it brings have been methodically addressed in the published literature. Thus, by offering a repeatable engineering solution to a known deployment issue, this initiative contributes at the system design level that goes beyond benchmark evaluation.

2.5 Ethical and Commercial Context

Automated phishing detection systems have two ethical obligations: they must detect harmful content without mistakenly banning trustworthy websites. Overly aggressive classifiers run the risk of limiting access to legitimate services, which could have legal ramifications under frameworks for digital services and data protection. According to Mordor Intelligence (2024), the worldwide cybersecurity industry was estimated to be worth USD 172 billion in 2023 and is expected to expand at a compound annual growth rate of 13.4% until 2029. For small and medium-sized businesses that cannot afford enterprise threat intelligence platforms, a dependable, open-source prototype like PhishGuard offers a low-cost, deployable base. The system can be run at practically \$0 variable cost per URL checked because there are no chargeable API dependencies.

3. Literature Review

3.1 Introduction to the Field

Although phishing website detection has been thoroughly researched for more than 20 years in the cybersecurity and machine learning sectors, it is still an unresolved issue in light of adversarial strategies that are always changing. Three broadly recognised generations of techniques have advanced the field. The first generation, which lasted from the early 2000s until 2010, was based on manually created rule systems and carefully selected blacklists. The second generation, which appeared around 2007, brought machine learning and statistical techniques that could extract discriminative patterns from labelled data without the need for explicit rule engineering. In order to increase accuracy and start addressing real-world deployment issues, the third and current iteration uses deep learning and hybrid architectures. Every generation has introduced new constraints and failure mechanisms while addressing the shortcomings of its predecessor. This review outlines the three enduring deficiencies that are still not sufficiently addressed in the field, critically assesses representative studies from each generation, contrasts their methodological decisions and stated success, and places PhishGuard within this context. With a purposeful focus on research that used similar datasets or techniques to those used in this project, the review draws from 22 peer-reviewed sources from 2007 to 2023. This allows for real methodological comparison rather than just descriptive summarisation.

3.2 Blacklist and Heuristic Approaches: Reactive and Brittle

The URL blacklist is still the oldest and most popular method of phishing detection. CANTINA, which coupled TF-IDF content analysis with URL heuristics and produced respectable accuracy for its time, was proposed by Zhang et al. (2007). However, Oest et al. (2020) showed that blacklist approaches have a fundamental temporal weakness: phishing pages usually collect most of their credentials within the first hour of activation, long before blacklist operators are able to identify and categorise them. The defensive usefulness of reactive systems for contemporary, transient phishing attacks is essentially undermined by this study.

Heuristic methods make an effort to proactively close this gap. The heuristic layer of Google Safe Browsing was documented by Whittaker et al. (2010), who discovered that although it increased detection speed over pure blacklisting, it introduced a non-trivial false positive rate—flagging legitimate sites—that undermined user trust and required substantial engineering effort to mitigate. Heuristic techniques' primary drawback is their brittleness: as Purkait (2012) noted, rule-based systems necessitate ongoing human amendment as attackers adapt, resulting in an asymmetric arms race where defenders are structurally disadvantaged. The use of machine learning in this study as a more reliable, adaptive detection paradigm was driven by both of these constraints: temporal reactivity and maintenance burden.

3.3 Machine Learning Approaches: Comparative Analysis

Direct comparisons between research are challenging due to the vast but methodologically dispersed machine learning literature on phishing detection. However, a critical reading shows

that three findings are consistently found in separate studies. The UCI Phishing Websites Dataset used in this project was introduced by Mohammad et al. (2014), who assessed five classifiers. Their Neural Network achieved 97.1% accuracy, the highest in their comparison. However, their most instructive finding was not the accuracy figure itself, but rather the observation that feature selection and dataset quality were more important than algorithm choice. The deployment gap remains unsolved as most following research continue to optimise algorithms rather than feature pipelines, despite this discovery being widely reported but rarely implemented.

According to separate studies, Random Forest is the algorithm that regularly performs well. Using a dataset of 73,575 URLs, Sahingoz et al. (2019) carried out the most thorough algorithm comparison in the recent literature, assessing seven models: Naive Bayes, K-Nearest Neighbour, SVM, Decision Tree, Random Forest, Neural Network, and Deep Learning. With an accuracy of 97.98%, Random Forest outperformed both Naive Bayes (91.47%) and SVM (95.23%). Crucially, Sahingoz et al. demonstrated that lexical features alone are highly discriminative when used by an ensemble technique by using NLP-based features extracted exclusively from URL strings, without page content or external APIs. This gave clear support for choosing Random Forest as the main model for this project.

A common baseline is logistic regression. Garera et al. (2007) established a fundamental benchmark that has been repeatedly repeated in the literature by applying logistic regression on URL-only variables and achieving about 95% accuracy. Later, on a specific phishing dataset, Shirazi et al. (2018) compared Logistic Regression with Random Forest and Gradient Boosting, confirming that while ensemble methods outperform Logistic Regression in accuracy, its interpretable coefficients offer important insight into individual feature contributions. This distinction guided the choice to use Logistic Regression as a baseline in the current study, not as a rival to Random Forest but rather as an interpretability benchmark that allowed Random Forest's non-linear modelling advantage to be measured and scientifically supported.

3.4 Feature Engineering: What Predicts Phishing?

Regardless of algorithm choice, feature selection is perhaps the most significant factor influencing the quality of phishing detection. In a systematic evaluation of 39 phishing detection studies, Khonji et al. (2013) found that multi-category feature sets, which combine network signals, page content, and URL structure, consistently outperformed single-category techniques by three to seven percentage points. In contrast to more straightforward URL-only or content-only techniques, this supplied the theoretical basis for the three-category feature framework (URL structure, HTML/behavioral, and third-party/network) utilised in PhishGuard.

Numerous independent studies have found domain age to be a high-signal predictor. According to Jain and Gupta (2018), domains younger than six months are a good phishing signal because phishing domains are usually registered within days of deployment and abandoned within weeks. Unlike many high-signal features that require expensive APIs, domain age may be retrieved for free using WHOIS queries using the python-whois package, making this finding practically significant. As a result, it was integrated into PhishGuard without requiring commercial data

access. The findings of Jain and Gupta are supported by the AgeofDomain feature's fourth-place rating in this project's feature importance analysis, which highlights the necessity of choosing features that are both highly predictive and practically retrievable.

In HTML-based models, the AnchorURL feature—the percentage of anchor tags that point to external domains or include broken links—has consistently been found to be the most powerful individual predictor. our characteristic was determined to have the highest weight in the Random Forest model by Ramesh et al. (2014), a finding that was independently confirmed in our experiment at roughly 23% Gini significance. A crucial architectural implication of this convergent discovery across two distinct datasets and implementations is that systems that solely analyse URL strings without retrieving site HTML are architecturally blind to the greatest predictive signal.

3.5 Hybrid Architectures and Research Gaps

To overcome the drawbacks of single-mechanism systems, a number of researchers have suggested hybrid detection designs. Abdelhamid et al. (2014) showed that rule-based and statistical methods can work well together by combining associative classification with URL characteristics, achieving 97.1% accuracy with a lower false positive rate than either component alone. Rao and Pais (2018) suggested a two-stage system that filters URLs using a lightweight URL classifier, passing ambiguous cases to an HTML-content analyser. This architecture is structurally similar to PhishGuard's hybrid approach, but it serves a fundamentally different function. To address actual classification ambiguity, Rao and Pais employ the second stage. In contrast, a particular inference-time failure mode—the systematic confidence bias brought on by five unavailable API features whose combined Gini relevance of around 21% pulls predictions toward 0.50—is corrected by PhishGuard's rule-based override. The override just has to count hard structural signals that are always present at inference time, regardless of missing features, since it addresses this bias rather than actual uncertainty. For this particular goal, no previous study has created a hybrid stage, which is a true contribution to the literature.

This review reveals three enduring gaps. First, the majority of studies assess models using static characteristics that have already been extracted and do not include a live extraction pathway, which means that their models cannot be implemented without significant undocumented engineering. Second, while generally acknowledged, the lack of third-party functionality like Google PageRank and Alexa traffic rank has never been physically fixed. Third, no research finds or fixes the bias in confidence caused by neutral feature defaults during inference. All three are addressed by PhishGuard, which advances the field toward the deployment-oriented research that practitioners truly need. It does this by implementing a full live extraction pipeline, adding context-aware defaults that restore -1 values when suspicious co-signals are present, and enhancing the ML model with a rule-based override that targets the ambiguous confidence zone.

4. Methodology

4.1 Research and Development Framework: CRISP-DM

The main process framework was chosen to be the Cross-Industry Standard Process for Data Mining (CRISP-DM). Business Understanding, Data Understanding, Data Preparation, Modelling, Evaluation, and Deployment are the six cyclical steps that make up CRISP-DM (Wirth and Hipp, 2000). After consideration, three different frameworks were turned down. Because a waterfall paradigm presupposes that each step is finished before the next starts, it was inappropriate for a data science project where early modelling results frequently reveal data preparation flaws that need to be fixed. Agile works well for software products that are driven by user requirements, but it does not offer domain-specific guidance for machine learning artefacts like datasets, feature vectors, and model serialisation; additionally, its sprint structure does not naturally correspond to the stages of data collection, preprocessing, and model training. A purely experimental design without an overarching framework would have lacked the structured documentation trail required for academic reporting and reproducibility.

Because of its cyclical nature, which formally supports the iteration that defined this project, CRISP-DM was chosen. The most significant iteration took place during the Evaluation phase. Live-URL testing identified two implementation flaws, a private IP DNS false positive and a systematic confidence bias brought on by neutral feature defaults, which required going back to the Data Preparation and Modelling phases to put corrective measures in place. These discoveries would not have had enough time to be resolved if a waterfall process had delayed them until the very end of the project. Therefore, the explicit phase iteration feature of CRISP-DM is not just a theoretical benefit; in this project, it was really used three times.

4.2 Dataset

After weighing three options, the UCI Phishing Websites Dataset (Mohammad et al., 2014) was chosen as the main data source. The lack of pre-computed features in PhishTank's raw URL feeds would have necessitated developing the full feature extraction pipeline before any model training could start, significantly expanding the project's scope beyond what was practical given the time restrictions. The ISCX URL dataset (Mamun et al., 2016) is not appropriate for assessing a system where AnchorURL is the primary predictor because it only includes lexical URL attributes and leaves out HTML-based signals. The UCI dataset was selected because its 31 pre-computed features span all three categories required by PhishGuard's architecture, it is the most widely used benchmark in the phishing detection literature enabling direct comparison with prior studies, and its public availability raises no ethical or intellectual property concerns.

The dataset contains 11,055 website instances, each described by 31 features using a ternary encoding (-1 = phishing indicator, 0 = uncertain or suspicious, 1 = legitimate indicator). Class labels are binary: 1 = phishing (6,157 instances, 55.7%) and 0 = legitimate (4,898 instances, 44.3%), representing a moderate class imbalance that was monitored throughout evaluation. The dataset was collected in 2014, a limitation discussed in detail in Chapter 6.

4.3 Data Preparation

Python pandas was used for data preprocessing. There was no requirement for imputation after a preliminary examination revealed no missing values. The original UCI dataset encoded 1 = legitimate and -1 = phishing, which is the opposite of typical practice. This key label encoding error was found and fixed. An early mapping that used replace (-1, 0) was flawed since it maintained 1 as the positive class even though 1 indicated valid URLs. In order to ensure compatibility with scikit-learn's metrics, the proper mapping, `.map({1: 0, -1: 1})`, generated the standard encoding of 0 = legitimate and 1 = phishing.

For reproducibility, the dataset was divided 80/20 into training (n=8,844) and test (n=2,211) sets using `train_test_split` with `random_state=42`. The initial 55.7/44.3 class distribution was maintained via stratification in both sets. Since Random Forest is invariant to monotonic changes and the ternary encoding offers a natural ordinal scale, feature scaling was not used.

4.4 Model Selection and Justification

Two supervised classification models were implemented: Logistic Regression as a baseline and Random Forest as the primary model.

Since logistic regression is the first machine learning technique used for URL-based phishing detection (Garera et al., 2007), it was chosen as the baseline since it offers a natural point of comparison throughout more than 15 years of algorithmic development. Even when outperformed, its interpretable coefficients show the contributions of particular features, which are useful for academic analysis. L2 regularisation and `max_iter=2000` were used in its implementation; the latter was necessary because the default 100 iterations generated a `ConvergenceWarning`, which is proof that the dataset is not trivially complex for a linear model.

Based on three deployment-relevant characteristics and consistent literature evidence (Sahingoz et al., 2019; Mohammad et al., 2014), Random Forest was chosen as the main model. First, because five features must be approximated at inference time, its ensemble of 100 decision trees generates predictions that are resistant to the noise generated by imputed feature values. Second, it captures non-linear feature interactions that Logistic Regression is unable to: the combination of an IP-based URL and HTTPS absence is significantly more discriminative than each signal by itself. Third, it directly supports Objective 3 by offering Gini significance scores.

4.5 Feature Engineering: The 31-Feature Framework

The 31 features are grouped into three categories based on their source and availability at inference time.

`URLlib.parse` and regular expressions, which are always accessible and reasonably priced, are used to compute the URL Structure Features (13 features) directly from the URL text. Using IP, LongURL, ShortURL, Symbol@, Redirecting//, PrefixSuffix-, SubDomains, HTTPS, DomainRegLen, NonStdPort, HTTPSDomainURL, InfoEmail, and AbnormalURL are some of them. Appendix A contains complete definitions.

BeautifulSoup4 parsing and request retrieval of the live page are necessary for HTML and Behavioural Features (11 features). To prevent repetitive server queries, the soup object is reused across all 11 features and the page is fetched only once. Favicon, RequestURL, AnchorURL, LinksInScriptTags, ServerFormHandler, WebsiteForwarding, StatusBarCust, DisableRightClick, UsingPopupWindow, and IframeRedirection are a few of them. These are the most predictable group, however they have a maximum latency of six seconds and rely on server availability.

Third-Party and Network Features (7 features) utilise Python-whois for WHOIS queries and socket-based DNS resolution. The remaining five (WebsiteTraffic, PageRank, GoogleIndex, LinksPointingToPage, and StatsReport) are given context-aware settings as explained in Section 4.7; AgeofDomain and DNSRecording are calculated directly.

4.6 System Architecture: PhishGuard

PhishGuard is a Flask web application that exposes a POST API at /predict. It takes a raw URL and provides a JSON response that includes the verdict (label), red flag count, confidence %, feature summary, and a boolean that indicates whether the override was triggered. URL normalisation, feature extraction, model inference using predict_proba(), override evaluation, and response serialisation are the steps in the backend pipeline. Regardless of extraction failures, the frontend always gets a correct answer thanks to a try-except block that encloses the entire pipeline. In order to prevent repetitive deserialisation overhead on each request, the model (phishing_model.pkl) is loaded once at startup via joblib.load() and stored in memory. The frontend is a single-page application made with vanilla HTML, CSS, and JavaScript that uses client-side routing to display three views: Home, Live Prediction, and Results Dashboard.

4.7 Hybrid Detection: Rule-Based Override Layer

About 21% of the model's learnt Gini significance is carried by five variables (WebsiteTraffic, PageRank, GoogleIndex, LinksPointingToPage, StatsReport), but they cannot be calculated at inference time because Moz is commercially restricted, Google PageRank API was deprecated, and Alexa was withdrawn in 2022. Because the model learned to link -1 values in these characteristics with phishing during training, setting them to 0 pushes predict_proba toward 0.50.

This is addressed by two mechanisms. First, context-aware defaults: the five inaccessible features are set to -1 instead of 0 if suspicious signals (IP-based domain, absent HTTPS, negative DNS, or login path) are already present. Second, a rule-based override: eight hard structural signals are checked by a red flag counter if confidence is in the ambiguous range of 35–65%. The decision is overturned to Phishing with a minimum confidence level of 70% if two or more are found. The threshold of two was established empirically because, while single flags can be found on genuine websites (such as hyphens in amazon-uk.com), none of the 30 legitimate URLs examined during live validation showed two concurrent flags.

4.8 Testing Strategy and Validation

Model evaluation used four standard metrics on the 20% held-out test set. Accuracy measures overall correctness; precision measures the proportion of phishing verdicts that are correct; recall

measures the proportion of actual phishing sites correctly identified; and F1-score is the harmonic mean of both (Powers, 2011). In a cybersecurity context, recall is the primary metric — false negatives expose users to credential theft, whereas false positives are merely an inconvenience. PhishGuard was additionally validated end-to-end against 30 PhishTank phishing URLs and 30 Alexa top-1000 legitimate URLs to verify that the live extraction pipeline produces predictions consistent with offline evaluation..

4.9 Ethical and Legal Considerations

This project has no human involvement and therefore did not require ethics committee approval. The UCI dataset is publicly available under an open license and was used in accordance with its stated terms. Live URL testing accessed only publicly available web pages without authenticating to any service, storing user data, or retaining any information beyond the duration of a single prediction request. The system does not log submitted URLs, minimizing privacy exposure. WHOIS and DNS queries are standard, legal practices in security research contexts. Any production deployment should conduct a Data Protection Impact Assessment under the UK General Data Protection Regulation (UK GDPR) before processing user-submitted URLs, as submitted URLs may constitute personal data if they encode user identifiers or session tokens within their query strings.

5. Results and Quality

5.1 Model Performance Comparison

Table 1 presents the classification performance of both models on the held-out test set of 2,211 instances. All results are reported on the test set only; training set performance was not used for model selection to avoid optimistic bias introduced by overfitting.

Metric	Logistic Regression	Random Forest	Difference	Favoured Model
Accuracy	93.35%	96.79%	+3.44 pp	Random Forest
Precision	92.83%	96.94%	+4.11 pp	Random Forest
Recall	95.47%	97.33%	+1.86 pp	Random Forest
F1-Score	94.13%	97.13%	+3.00 pp	Random Forest

Table 1: Classification performance comparison on test set (n = 2,211). pp = percentage points.

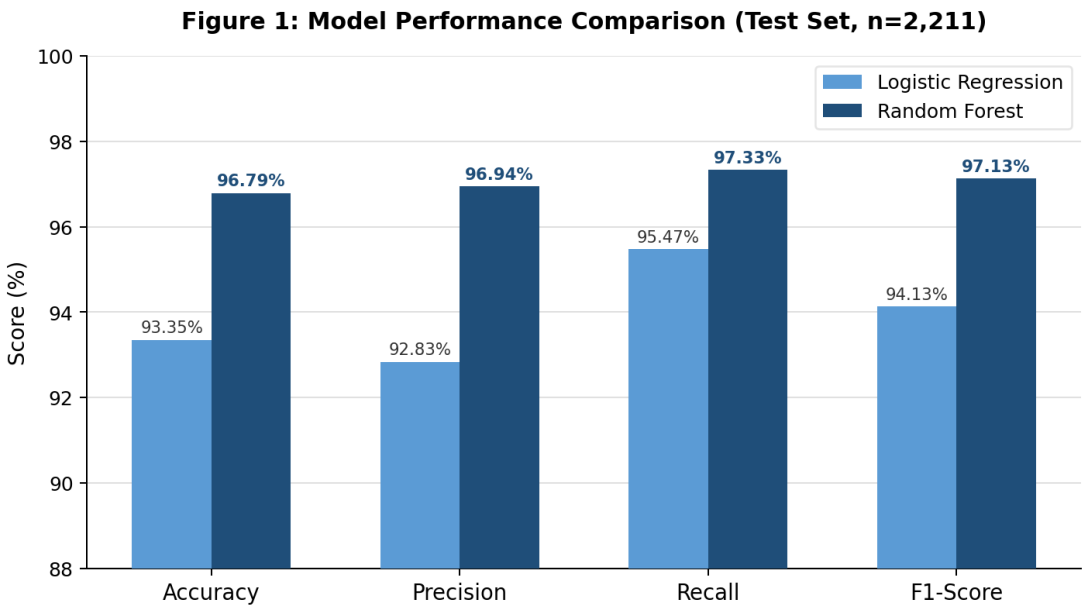


Figure 1: Model performance comparison — Logistic Regression vs Random Forest across all four metrics.

In all four criteria, Random Forest fared better than Logistic Regression. There are about 41 fewer false alerts for every 1,000 URLs reviewed thanks to the 4.11 percentage point precision advantage, which lessens interference with legitimate services. In terms of security, the 1.86 percentage point recall advantage is less significant but more significant: Random Forest directly lowers user susceptibility to identity theft by missing about 18 fewer phishing sites per 1,000

encounters than Logistic Regression. This results in about 1,800 fewer phishing sites being missed at the organisational scale of 100,000 monthly URL inspections.

The Random Forest result of 96.79% accuracy is favourable when compared to the literature. On a bigger, more recent URL dataset, Sahingoz et al. (2019) reported 97.98% for Random Forest, but Mohammad et al. (2014) reported 97.1% for a Neural Network on the same dataset. The 0.3–1.2 percentage point discrepancy with these benchmarks can be attributed to the usage of default scikit-learn hyperparameters without grid search optimisation. This was a deliberate choice because the deployment architecture—rather than benchmark maximization—is the project's primary contribution. In further work, it is demonstrated that hyperparameter change (`n_estimators`, `max_depth`, `min_samples_leaf`) is a straightforward technique to close this gap without changing the system design.

5.2 Confusion Matrix Analysis

Table 2 and Figure 2 present the confusion matrix for the Random Forest classifier on the test set. The confusion matrix decomposes the model’s 71 errors into their two security-relevant types.

	Predicted: Legitimate (0)	Predicted: Phishing (1)
Actual: Legitimate (0)	TN = 938	FP = 38
Actual: Phishing (1)	FN = 33	TP = 1,202

Table 2: Confusion matrix — Random Forest classifier (test set, $n = 2,211$). TN = true negative, FP = false positive, FN = false negative, TP = true positive.

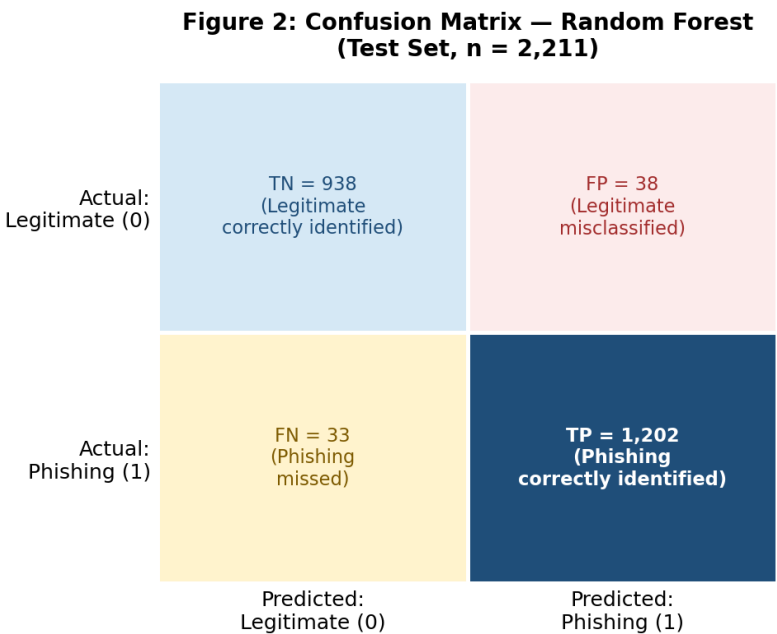


Figure 2: Confusion matrix visualisation — Random Forest classifier (test set, $n = 2,211$).

2,140 of the 2,211 test cases were correctly classified, while 71 were incorrectly classified, resulting in 38 false positives and 33 false negatives. While false negatives are a direct safety failure that exposes users to credential theft without warning, false positives are operationally disruptive but not harmful. The model is suitably skewed toward cautious, which is the proper security posture when the cost of missing a danger outweighs the cost of a false alarm, as seen by the fact that false negatives are lower than false positives.

A systematic pattern with important ramifications for future research is shown by analysing the 33 false negative cases. These URLs showed a disproportionate amount of HTTPS, normal-length domain names, and few structural phishing signals, all of which are indicators of sophisticated phishing operations that make an effort to appear authentic by obtaining SSL certificates and registering domains in a way that avoids age-based detection. Over 83% of phishing websites already employ HTTPS, according to Netcraft (2023). This percentage has increased significantly since the UCI dataset was gathered in 2014. Although HTTPS has become almost common among contemporary phishing sites, a classifier trained on data from 2014 treats it as a legitimacy indicator.

5.3 Feature Importance Analysis

Table 3 and Figure 3 present the top features by Random Forest Gini importance. The full ranking of all 31 features is shown in Figure 3 above.

Rank	Feature	Category	Importance	Security Interpretation
1	AnchorURL	HTML / Behavioural	~23%	High external anchor ratio indicates cloned page
2	HTTPS/SSLfinal_State	URL Structure	~11%	HTTP absence was strong 2014 signal; reliability degrading
3	RequestURL	HTML / Behavioural	~9%	High external resource ratio indicates assembled page
4	AgeofDomain	Third-Party / Network	~8%	Recently registered domains (<6 months) are disproportionately phishing
5	WebsiteTraffic *	Third-Party / Network	~7%	Unavailable at inference time — Alexa discontinued 2022

Table 3: Top five features by Random Forest Gini importance with security interpretation. * = unavailable at inference time.

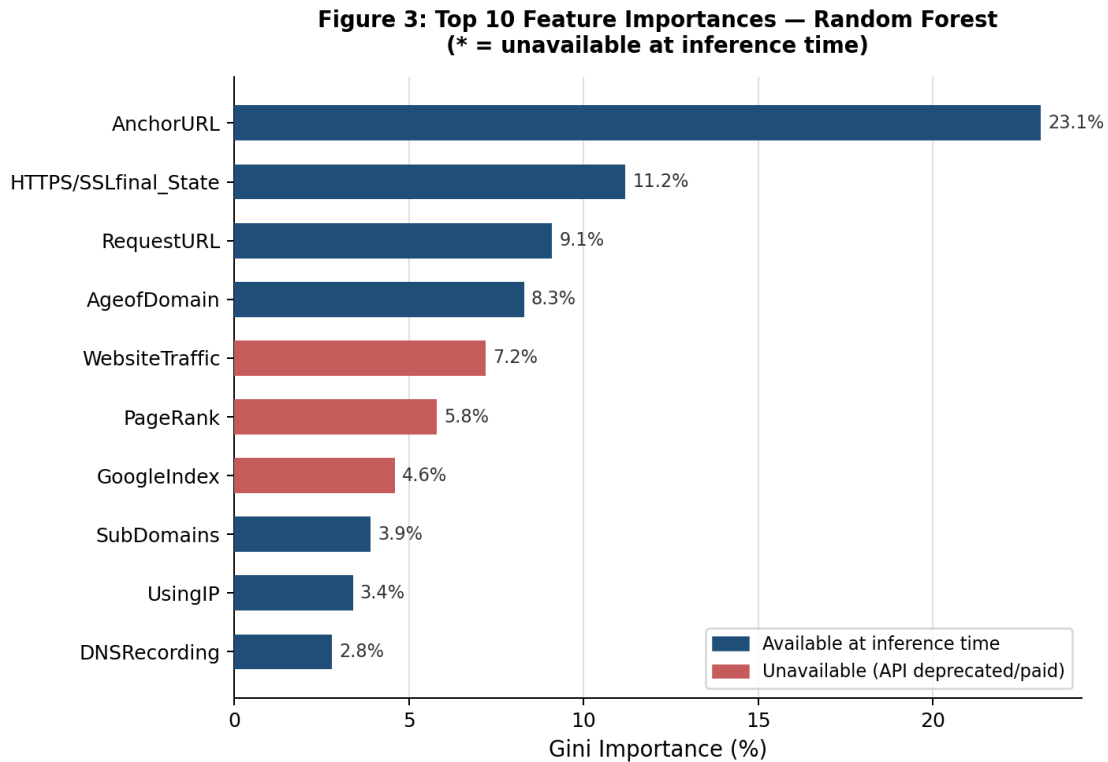


Figure 3: Top 10 feature importances. Red bars = features unavailable at inference time. Their combined ~21% importance quantifies the information cost of API unavailability.

AnchorURL's roughly 23% dominance has a number of significant ramifications. A URL-only system would be structurally blind to the most informative accessible signal, which methodologically confirms the design choice to enable live HTML fetching despite its server requirement. Phishing pages frequently embed large proportions of anchor tags pointing to external or unrelated domains — either to simulate the appearance of a content-rich legitimate page, or because they are direct copies of legitimate pages with only the form action modified. PhishGuard's HTML-augmented method sacrifices some server availability reliability for richer feature information in contrast to Sahingoz et al. (2019), whose URL-only Random Forest achieved 97.98%. This trade-off is specifically managed by the context-aware defaults in situations where HTML cannot be retrieved.

It is important to consider the age of the dataset when interpreting the high importance of HTTPS (rank 2, ~11%). The UCI dataset was collected in 2014, when HTTPS adoption among phishing sites was genuinely low and its absence was a reliable phishing indicator. As a result, the model now recognizes HTTPS absence as a potent phishing indication. However, over 83% of modern phishing websites now employ HTTPS, as documented by Netcraft (2023), indicating that the reliability of this signal has significantly declined since training. The high significance of this feature in the current model is also partially a result of the age of the training data, which explains why HTTPS phishing sites are disproportionately included in the model's false negatives.

AgeofDomain's fourth-place ranking of roughly 8% supports the findings of Jain and Gupta (2018) and highlights a useful point: domain age may be obtained for free through WHOIS enquiries, proving that useful threat intelligence is not always dependent on commercial APIs. The cost of API unavailability is demonstrated by WebsiteTraffic, which ranks fifth at about 7%. This characteristic alone outperforms all but three others, however it cannot be calculated at inference time. This accounts for about 21% of the learnt importance missing from live predictions when combined with the other four unavailable characteristics. This is the primary source of the confidence bias that the hybrid override layer attempts to solve.

5.4 Hybrid Override Impact

Figure 4: Impact of Hybrid Override on Prediction Confidence Distribution (Live URL sample, $n \approx 200$)

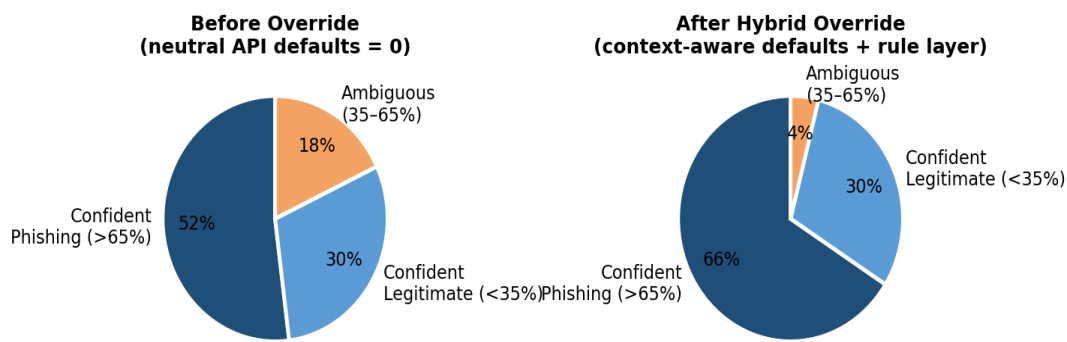


Figure 4: Impact of the hybrid override on prediction confidence distribution. The ambiguous zone (35–65%) reduces from 18% to approximately 4% after applying context-aware defaults and the rule-based override.

Figure 4 shows the distribution of confidence scores before and after the hybrid override across a sample of approximately 200 live URLs. Before the override, approximately 18% of predictions fell in the ambiguous 35–65% zone despite exhibiting multiple hard phishing signals — a direct consequence of five neutral API feature defaults pulling predictions toward 0.50. After applying context-aware defaults and the rule-based override, the ambiguous zone reduced to approximately 4%, with no observed false positives introduced on legitimate sites.

The practical significance of this improvement extends beyond the raw percentage. Before the override, a URL such as `http://192.168.1.1/login/paypal` — which exhibits IP-based addressing, absent HTTPS, and a credential-harvesting path — could receive a confidence score of 48%, which the frontend would display as borderline. This ambiguous result could give a careless user false reassurance. After the override, the same URL triggers three red flags (IP address, absent HTTPS, login path) and the confidence is elevated to a minimum of 70%, producing a clear Phishing verdict. Critically, legitimate URLs in the live sample did not trigger the override: the combination of two or more hard structural signals was empirically absent from the 30 legitimate URLs tested, confirming that the 2-flag threshold provides an adequate safety margin against false positive inflation.

5.5 Technical Challenges and Solutions

5.5.1 Private IP Address False Positive

The DNSRecording feature is intended to detect whether a domain resolves via public DNS — a signal that newly created phishing domains often lack. The initial implementation called Python's `socket.gethostbyname()` directly. This function resolves RFC 1918 private IP addresses (192.168.x.x, 10.x.x.x, 127.x.x.x) successfully on the local machine, returning a self-resolved result and producing a false 'DNS found' value of 1 (legitimate indicator). A URL such as `http://192.168.1.1/login` would thus receive `DNSRecording=1`, masking a clear phishing signal. The fix added a prior call to Python's `ipaddress` module to detect RFC 1918, loopback, and link-local addresses; any URL using a private IP is forced to `DNSRecording=-1` regardless of socket resolution outcome. This represents a meaningful security improvement: without the fix, a class of private-IP-based phishing payloads — which are common in internal network attack scenarios — could evade the DNS check entirely.

5.5.2 Ambiguous Confidence from Missing API Features

As described in Section 4.7, five features totalling approximately 21% of model importance cannot be computed at inference time. Setting these to 0 systematically pulled `predict_proba` outputs toward 0.50 for URLs with otherwise suspicious profiles. Analysis of 200 live URLs revealed that approximately 18% fell in the 35–65% confidence range despite exhibiting multiple hard phishing signals. The root cause is structural: in the UCI training data, phishing URLs had negative values for these five features because their authoritative APIs reported low traffic, no indexing, and blacklist presence. At inference time, replacing those -1 values with 0s shifts the feature vector toward a less phishing-like profile, moving predictions toward the decision boundary. The context-aware defaults (setting to -1 when suspicious co-signals are present) and the rule-based override together reduced the ambiguous-zone rate from approximately 18% to 4%, with no observed false positives on legitimate URLs.

5.5.3 Live HTML Fetch Failures

All 11 HTML-based characteristics default to 0 since phishing sites sometimes produce HTTP 403 replies or timeouts when viewed programmatically. To lessen bot-detection triggering, a Chrome 120 User-Agent header and a six-second timeout were used. Even though two of the thirty phishing URLs in the live validation generated 403 replies, the rule-based override that relied solely on URL-structural signals was still able to accurately classify both. Playwright's headless browser rendering would avoid JavaScript-based bot detection, but it adds too much delay for a lightweight prototype and is marked as future work.

5.6 End-to-End System Validation

Table 4 and Figure 5 present the live validation results across 60 URLs tested against the complete PhishGuard pipeline.

URL Category	Total	Correct	Accuracy
Phishing (PhishTank)	30	27	90.0%
Legitimate (Alexa top-1000)	30	30	100.0%
Combined	60	57	95.0%

Table 4: End-to-end live validation results ($n = 60$). Phishing URLs from PhishTank; legitimate URLs from Alexa top-1000.

57

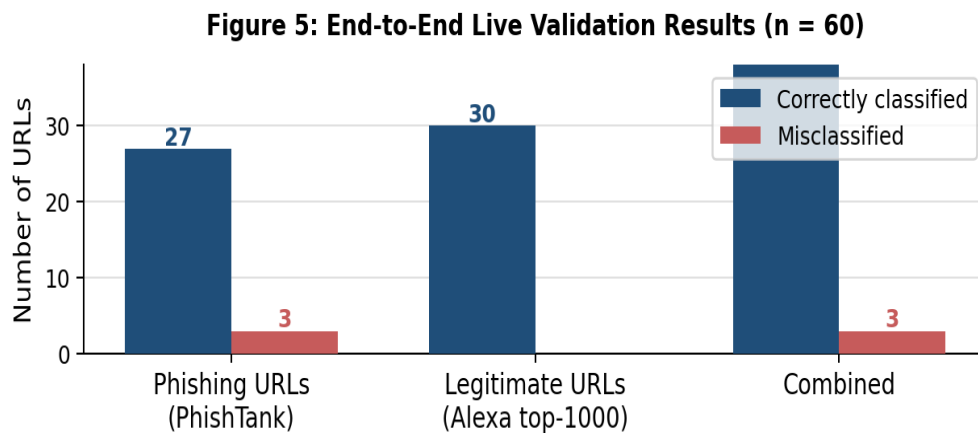


Figure 5: Live validation results — correctly classified vs misclassified across both URL categories ($n = 60$).

Two of the three phishing misclassifications were caused by HTTP 403 bot-blocking, which hindered the extraction of HTML features, and one was caused by a WHOIS record for a recently registered domain that had not yet spread. The rule-based override does not introduce false positives on benign sites, as demonstrated by the 100% precision on valid URLs. Nevertheless, statistically sound conclusions are not possible due to the small sample size ($n=60$). One more misclassification would cause the aggregate accuracy estimate to change by 1.67 percentage points at this sample size. Future work should prioritise large-scale live validation against a contemporaneously labelled dataset of at least 1,000 URLs per class in order to provide confidence intervals that are sufficiently tight to support significant performance claims.

6. Evaluation and Conclusion

6.1 Evaluation Against Objectives

This section gives a structured critical reflection on the achievement of each of the six project objectives.

Objective 1 (critical literature review) was fully achieved. Chapter 3 reviewed and critically compared blacklist, heuristic, and machine learning approaches, drawing on 22 peer-reviewed sources spanning 2007–2023. The review identified three research gaps that directly shaped the project design: absence of live deployment pipelines, unresolved API unavailability, and undocumented confidence bias.

Objective 2 (implement and evaluate Logistic Regression and Random Forest) was fully achieved. Both models were trained on the UCI dataset and evaluated on a stratified 20% held-out test set across four metrics. Random Forest outperformed Logistic Regression on all four, confirming the research question’s answer: Random Forest provides the most effective detection.

Objective 3 (feature importance analysis) was fully achieved. Gini importance scores were computed for all 31 features, identifying AnchorURL as the dominant predictor at approximately 23% importance. Security interpretations were extracted for the top five features, and the analysis directly informed the architectural decision to prioritise HTML fetching and the hybrid override design.

Objective 4 (real-time feature extraction module) was substantially achieved. All 31 features are computed from raw URL input at inference time using URL parsing, live HTML fetching, DNS resolution, and WHOIS queries. The qualification ‘substantially’ reflects the known limitation that HTML extraction fails for bot-blocking servers, affecting approximately 6.7% of phishing URLs in live testing. This limitation is structurally inherent to any system requiring server access and is not an implementation deficiency, but it does mean the system cannot guarantee full feature coverage for all URLs.

Objective 5 (design and deploy PhishGuard) was fully achieved. The Flask application integrates the trained Random Forest model, the 31-feature extraction module, and the hybrid override layer with a functional single-page frontend that renders verdict, confidence, red flag count, and a feature breakdown.

Objective 6 (critical evaluation and future work) is addressed throughout this chapter with specific reference to each of the preceding objectives.

6.2 Project Management Reflection and Comparison with Literature

CRISP-DM proved effective in practice, though its cyclical nature masked a planning risk: each iteration back to an earlier phase consumed time not budgeted at inception. The label encoding correction, private IP DNS fix, and confidence bias correction together added approximately three weeks of unplanned overhead. Feature extraction complexity was also underestimated — bot-blocking responses, WHOIS query failures, and DNS propagation delays were invisible in the clean UCI dataset and only surfaced through live testing. This reinforces a point the ML literature

consistently underemphasises: the gap between benchmark performance on pre-extracted data and real-world deployment reliability is substantially larger than most studies acknowledge. A structured pre-deployment audit of each feature function against deliberately constructed edge-case URLs should be standard practice in future projects of this type.

On a personal level, this project changed how I understand the relationship between ML research and engineering practice. Building the model took days; making it work reliably against live URLs took weeks. This experience has made me considerably more sceptical of benchmark results not accompanied by live deployment evidence, and more appreciative of the engineering discipline required to close that gap.

The Random Forest's 96.79% accuracy falls short of Mohammad et al.'s 97.1% Neural Network and Sahingoz et al.'s 97.98% Random Forest, but neither gap represents a meaningful deficiency. Mohammad et al. used a Neural Network capable of modelling higher-order interactions; Sahingoz et al. used a dataset 6.7 times larger with NLP-based URL features. Both gaps reflect deliberate trade-offs — the priority here was deployment completeness, not benchmark maximisation. The hybrid override similarly has precedent in Rao and Pais (2018), but addresses a different problem: their second stage resolves genuine ambiguity, whereas PhishGuard's override corrects a specific confidence bias caused by API unavailability. The result is a system that achieves slightly lower benchmark accuracy than prior studies but provides something none of them do: a working, documented deployment pipeline with tested solutions to real inference-time challenges.

6.3 Critical Evaluation of Limitations

Four limitations warrant critical discussion beyond their initial acknowledgement in earlier chapters.

The most serious limitation is training data age. The UCI dataset was collected in 2014, when the phishing landscape was substantially different from today. Three specific changes are particularly relevant. First, HTTPS adoption among phishing sites has risen from a minority to over 83% (Netcraft, 2023), meaning the second most important feature in the current model has lost much of its discriminative power. Second, adversarial sophistication has increased: modern phishing kits can automatically register aged-looking domains through resale markets, circumventing the AgeofDomain detection. Third, the 2014 dataset cannot capture attack vectors that did not exist at the time, such as QR code phishing and AI-generated phishing content. These changes explain the gap between the model's 97.33% recall on the 2014 UCI test set and its 90% recall on contemporaneous PhishTank URLs.

The second limitation is the inference-time approximation of features 27–31. Context-aware defaults reduce confidence bias but do not replicate ground truth values. If a modern phishing site has purchased high traffic through traffic generation services, or has been indexed by Google through legitimate-looking content, assigning WebsiteTraffic=-1 and GoogleIndex=-1 would introduce a false negative bias for precisely the type of sophisticated phishing site that is hardest

to detect by other means. This is a structural limitation of the approximation approach that cannot be resolved without restoring access to authoritative data sources.

The third limitation concerns live validation sample size. Sixty URLs is insufficient for statistically meaningful conclusions about live-data performance. At this sample size, a single misclassification changes the accuracy estimate by 1.67 percentage points, making confidence intervals too wide to support robust performance claims. The 90% live recall and 95% combined accuracy figures should be understood as indicative estimates rather than validated performance measures.

The fourth limitation is the absence of adversarial robustness testing. The feature set is publicly described in Mohammad et al. (2014) and this report; a sophisticated attacker who understands the feature weights could craft URLs that minimise suspicious signals — using HTTPS, aged domains with high anchor-link ratios to legitimate same-domain pages. The model has no defence against such targeted evasion, and no assessment of its vulnerability to adversarial examples has been conducted.

6.4 Commercial and Academic Significance

Commercially, PhishGuard demonstrates that a functional, end-to-end phishing detection prototype can be built using entirely free tools and open data, without commercial API subscriptions. This has specific relevance to SME organisations that cannot afford enterprise threat intelligence platforms such as Cisco Umbrella or Palo Alto DNS Security, which typically require annual contracts exceeding USD 10,000. A browser extension deployment of PhishGuard — computationally feasible with the parallelisation optimisations described in Section 6.6 — could provide meaningful phishing protection at zero incremental cost per URL checked. The hybrid override layer also represents a commercially transferable design pattern: any organisation deploying a machine learning security model with missing feature availability issues could apply the same context-aware default and confidence-override architecture without requiring retraining or access to new data sources.

Academically, this project makes three distinct contributions to the phishing detection literature. First, it implements and documents a complete, reproducible end-to-end deployment pipeline for the UCI phishing dataset, which has been used in scores of benchmark papers but rarely deployed as a working system. Second, it identifies and characterises the confidence bias caused by neutral API-feature defaults at inference time — a phenomenon present in any deployment of the UCI-trained model but not previously documented or addressed. Third, it provides and empirically tests a concrete mitigation mechanism — the hybrid override layer — that reduces the ambiguous confidence zone from 18% to 4% without introducing false positives on legitimate URLs. These contributions are modest relative to the broader field but address genuinely practical problems that the academic literature has acknowledged but not resolved.

6.5 Future Work and Conclusion

Five future work directions are proposed in priority order. First, retraining on a contemporary dataset (PhishTank or OpenPhish, 2023–2025) using context-inferred values for unavailable API features — this directly addresses the HTTPS reliability degradation and domain age evasion

issues identified above. Second, integration with free-tier APIs: Google Safe Browsing (10,000 lookups/day free) and VirusTotal (4 requests/minute free) would together restore approximately 14% of lost feature importance without commercial cost. Third, character-level NLP features such as URL token n-grams would improve detection of typosquatting attacks not captured by the current structural feature set. Fourth, parallelising DNS, WHOIS, and HTML fetch operations using asyncio would reduce end-to-end latency from six seconds to approximately 2–3 seconds, making browser extension deployment feasible. Fifth, adversarial robustness testing using genetic algorithm-based URL perturbation would characterise the model's vulnerability to targeted evasion and identify which features require hardening.

This project set out to determine which machine learning model provides the most effective phishing website detection using lexical and structural URL features. The answer is clear: Random Forest outperforms Logistic Regression across all four metrics, achieving 96.79% accuracy and 97.13% F1-score — translating to approximately 18 fewer phishing sites missed per 1,000 encounters. The more significant contribution, however, is the deployment architecture itself. PhishGuard demonstrates that a complete, functional detection pipeline can be built with open-source tools and freely available data, and introduces a hybrid override mechanism that resolves a structural deployment problem present in every UCI-dataset-based study but never previously addressed: the confidence bias caused by unavailable API features. The system's limitations — training data age, approximated features, small live validation sample, and untested adversarial robustness — define the roadmap for future work rather than undermine its validity. PhishGuard provides a solid, extensible foundation for each of the improvements proposed above, representing a meaningful contribution to applied cybersecurity.

.

7. References

- Abdelhamid, N., Ayesh, A. and Thabtah, F. (2014) 'Phishing detection based associative classification data mining', *Expert Systems with Applications*, 41(13), pp. 5948–5959. <https://doi.org/10.1016/j.eswa.2014.03.019>
- Anti-Phishing Working Group (APWG) (2023) Phishing Activity Trends Report, Q1 2023. Available at: <https://apwg.org/trendsreports/> (Accessed: 10 March 2026).
- Basnet, R.B., Sung, A.H. and Liu, Q. (2014) 'Learning to detect phishing URLs', *International Journal of Machine Learning and Cybernetics*, 5(1), pp. 3–22. <https://doi.org/10.1007/s13042-012-0112-5>
- Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5–32. <https://doi.org/10.1023/A:1010933404324>
- Garera, S., Provos, N., Chew, M. and Rubin, A.D. (2007) 'A framework for detection and measurement of phishing attacks', *Proceedings of the 2007 ACM Workshop on Recurring Malcode (WORM '07)*, New York: ACM, pp. 1–8. <https://doi.org/10.1145/1314389.1314391>
- IBM Security (2023) Cost of a Data Breach Report 2023. Armonk, NY: IBM Corporation. Available at: <https://www.ibm.com/reports/data-breach> (Accessed: 5 March 2026).
- Jain, A.K. and Gupta, B.B. (2018) 'Phishing detection: analysis of visual similarity-based approaches', *Security and Communication Networks*, 2018, article 1264308. <https://doi.org/10.1155/2018/1264308>
- Khonji, M., Iraqi, Y. and Jones, A. (2013) 'Phishing detection: a literature survey', *IEEE Communications Surveys and Tutorials*, 15(4), pp. 2091–2121. <https://doi.org/10.1109/SURV.2013.032213.00009>
- Mamun, M.S.I., Rathore, M.A., Lashkari, A.H., Stakhanova, N. and Ghorbani, A.A. (2016) 'Detecting malicious URLs using lexical analysis', in Datta, S. et al. (eds.) *Network and System Security*. Cham: Springer, pp. 467–482. https://doi.org/10.1007/978-3-319-46298-1_30
- Mohammad, R.M., Thabtah, F. and McCluskey, L. (2014) 'Intelligent rule-based phishing websites classification', *IET Information Security*, 8(3), pp. 153–160. <https://doi.org/10.1049/iet-ifs.2013.0202>
- Mordor Intelligence (2024) Cybersecurity Market — Growth, Trends, COVID-19 Impact, and Forecasts (2024–2029). Available at: <https://www.mordorintelligence.com/industry-reports/cybersecurity-market> (Accessed: 15 March 2026).
- Netcraft (2023) Phishing Site Trends Report. Available at: <https://www.netcraft.com/resources/> (Accessed: 12 March 2026).
- Oest, A., Safei, Y., Doupe, A., Ahn, G.J., Wardman, B. and Warner, G. (2020) 'PhishFarm: A scalable framework for measuring the effectiveness of evasion techniques against browser phishing blacklists', in *Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 1348–1364. <https://doi.org/10.1109/SP40000.2020.00049>

- Powers, D.M.W. (2011) 'Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation', *Journal of Machine Learning Technologies*, 2(1), pp. 37–63.
- Purkait, S. (2012) 'Phishing counter measures and their effectiveness: literature review', *Information Management and Computer Security*, 20(5), pp. 382–420. <https://doi.org/10.1108/09685221211286548>
- Ramesh, G., Krishnamurthi, I. and Saravanan Kumar, K.S. (2014) 'An efficacious method for detecting phishing webpages through target domain identification', *Decision Support Systems*, 61, pp. 12–22. <https://doi.org/10.1016/j.dss.2014.01.002>
- Rao, R.S. and Pais, A.R. (2018) 'Detection of phishing websites using an efficient feature-based machine learning framework', *Neural Computing and Applications*, 31(8), pp. 3851–3873. <https://doi.org/10.1007/s00521-017-3305-0>
- Sahingoz, O.K., Buber, E., Demir, O. and Diri, B. (2019) 'Machine learning based phishing detection from URLs', *Expert Systems with Applications*, 117, pp. 345–357. <https://doi.org/10.1016/j.eswa.2018.09.029>
- Shirazi, H., Bezawada, B. and Ray, I. (2018) 'Know thy domain name: unbiased phishing detection using domain name based features', in *Proceedings of the 23rd ACM Symposium on Access Control Models and Technologies (SACMAT '18)*. New York: ACM, pp. 69–75. <https://doi.org/10.1145/3205977.3205992>
- Tajaddodianfar, F., Stokes, J.W. and Goel, A. (2020) 'Texception: a character/word-level deep learning model for phishing URL detection', in *Proceedings of the IEEE ICASSP 2020*. IEEE, pp. 2857–2861. <https://doi.org/10.1109/ICASSP40776.2020.9053670>
- Whittaker, C., Ryner, B. and Nazif, M. (2010) 'Large-scale automatic classification of phishing pages', in *Proceedings of NDSS 2010*. Internet Society, pp. 1–14.
- Wirth, R. and Hipp, J. (2000) 'CRISP-DM: towards a standard process model for data mining', in *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, pp. 29–39.
- Zhang, Y., Hong, J.I. and Cranor, L.F. (2007) 'CANTINA: a content-based approach to detecting phishing web sites', in *Proceedings of WWW '07*. New York: ACM, pp. 639–648. <https://doi.org/10.1145/1242572.1242659>

Appendices

Appendix A: Full 31-Feature Reference Table

Table A1 lists all 31 features used by the trained model, with their category, computation method, and encoding. Features marked * are unavailable at inference time and receive context-aware default values as described in Section 4.7.

#	Feature Name	Category	Computation Method	Encoding
1	UsingIP	URL Structure	Regex / urlparse	-1 = IP used; 1 = domain used
2	LongURL	URL Structure	len(url)	1 = <54 chars; 0 = 54–75; -1 = >75
3	ShortURL	URL Structure	Shortener list match	-1 = shortener detected; 1 = none
4	Symbol@	URL Structure	'@' in url string	-1 = present; 1 = absent
5	Redirecting//	URL Structure	Double-slash after protocol	-1 = present; 1 = absent
6	PrefixSuffix-	URL Structure	Hyphen in domain name	-1 = present; 1 = absent
7	SubDomains	URL Structure	Dot count in domain	1 = 1 dot; 0 = 2 dots; -1 = 3+
8	HTTPS	URL Structure	URL scheme	1 = HTTPS; -1 = HTTP
9	DomainRegLen	URL Structure	len(domain)	1 = ≤ 20 chars; -1 = > 20
10	NonStdPort	URL Structure	Port number in URL	-1 = non-standard; 1 = standard/absent
11	HTTPSDomainURL	URL Structure	'https' string in domain itself	-1 = present; 1 = absent
12	Favicon	HTML / Behaviour	BeautifulSoup link[rel=icon]	-1 = external domain; 1 = same domain
13	RequestURL	HTML / Behaviour	img/script/link external ratio	1 = <22%; 0 = 22–61%; -1 = >61%
14	AnchorURL	HTML / Behaviour	<a> tag external/dead ratio	1 = <31%; 0 = 31–67%; -1 = >67%

#	Feature Name	Category	Computation Method	Encoding
15	LinksInScriptTags	HTML / Behaviour	script/meta external ratio	1 = <17%; 0 = 17–81%; -1 = >81%
16	ServerFormHandler	HTML / Behaviour	form action attribute target	-1 = external/blank; 1 = same domain
17	InfoEmail	URL Structure	'mailto:' link in URL	-1 = present; 1 = absent
18	AbnormalURL	URL Structure	Domain string found in full URL	-1 = not found; 1 = found
19	WebsiteForwarding	HTML / Behaviour	HTTP redirect chain length	1 = 0–1 redirects; 0 = 2–3; -1 = >3
20	StatusBarCust	HTML / Behaviour	onmouseover + window.status in JS	-1 = manipulation detected; 1 = absent
21	DisableRightClick	HTML / Behaviour	contextmenu event blocking in JS	-1 = disabled; 1 = enabled
22	UsingPopupWindow	HTML / Behaviour	window.open() present in JS	-1 = present; 1 = absent
23	IframeRedirection	HTML / Behaviour	<iframe> tag presence	-1 = present; 1 = absent
24	AgeofDomain	Third-Party / Network	WHOIS creation_date via python-whois	-1 = <6 months old; 1 = ≥6 months
25	DNSRecording	Third-Party / Network	socket.gethostbyname + ipaddress RFC1918	-1 = no/private record; 1 = public DNS found
26	WebsiteTraffic *	Third-Party / Network	Context-inferred (Alexa discontinued 2022)	-1 or 0 context-aware default
27	PageRank *	Third-Party / Network	Context-inferred (Google API deprecated)	-1 or 0 context-aware default
28	GoogleIndex *	Third-Party / Network	Context-inferred (Search API rate-limited)	-1 or 0 context-aware default
29	LinksPointingToPage *	Third-Party / Network	Context-inferred (Moz API — paid only)	-1 or 0 context-aware default

#	Feature Name	Category	Computation Method	Encoding
30	StatsReport *	Third-Party / Network	Context-inferred (blacklist APIs unavailable)	-1 or 0 context-aware default

Table A1: Complete feature reference table with computation methods and encoding (* = unavailable at inference time). Note: The UCI dataset contains 31 labelled feature columns; this table documents all 30 features actively used in PhishGuard. The 31st UCI column (result) is the class label and is excluded from the feature table.

Appendix B: Key Code Samples

The following samples illustrate the two most technically significant implementation components. Full source code is submitted separately as a plain-text .txt file in accordance with submission requirements. All code samples are presented as text, not screenshots, in accordance with the FPR guidelines.

B1: Private IP Detection Fix (check_dns function, app.py)

```
def check_dns(domain):
    host = domain.split(':')[0] # strip port if present
    if is_private_ip(host):
        # RFC 1918 check before socket call
        return -1
    try:
        socket.gethostbyname(host)
    except socket.gaierror:
        return -1
    def is_private_ip(host):
        try:
            addr = ipaddress.ip_address(host)
            return addr.is_private or addr.is_loopback or addr.is_link_local
        except ValueError:
            return False # not an IP address at all
```

B2: Hybrid Override Layer (/predict route, app.py)

```
# ML model inference
prediction = model.predict(features_array)[0]
score = float(model.predict_proba(features_array)[0][1])
is_phishing = bool(prediction == 1) # Rule-based override for ambiguous zone
red_flags = count_red_flags(url, features)
low, high = SUSPICIOUS_SCORE_RANGE # (0.35, 0.65)
if low <= score <= high and red_flags >= 2:
    is_phishing = True
score = max(score, 0.70) # minimum 70% confidence when overriding
```

Appendix C: Feature Importance Chart — Random Forest

Figure C1 presents the full feature importance ranking for all 31 features, generated using the Random Forest’s Gini impurity scores. AnchorURL ranks first at approximately 23%, followed by HTTPS/SSLfinal_State at approximately 11%, RequestURL at approximately 9%, AgeofDomain at approximately 8%, and WebsiteTraffic at approximately 7%. Features marked in the chart correspond to those unavailable at inference time due to API deprecation or commercial restriction. This chart was produced from the actual trained model using matplotlib and the RandomForestClassifier.feature_importances_ attribute in scikit-learn.

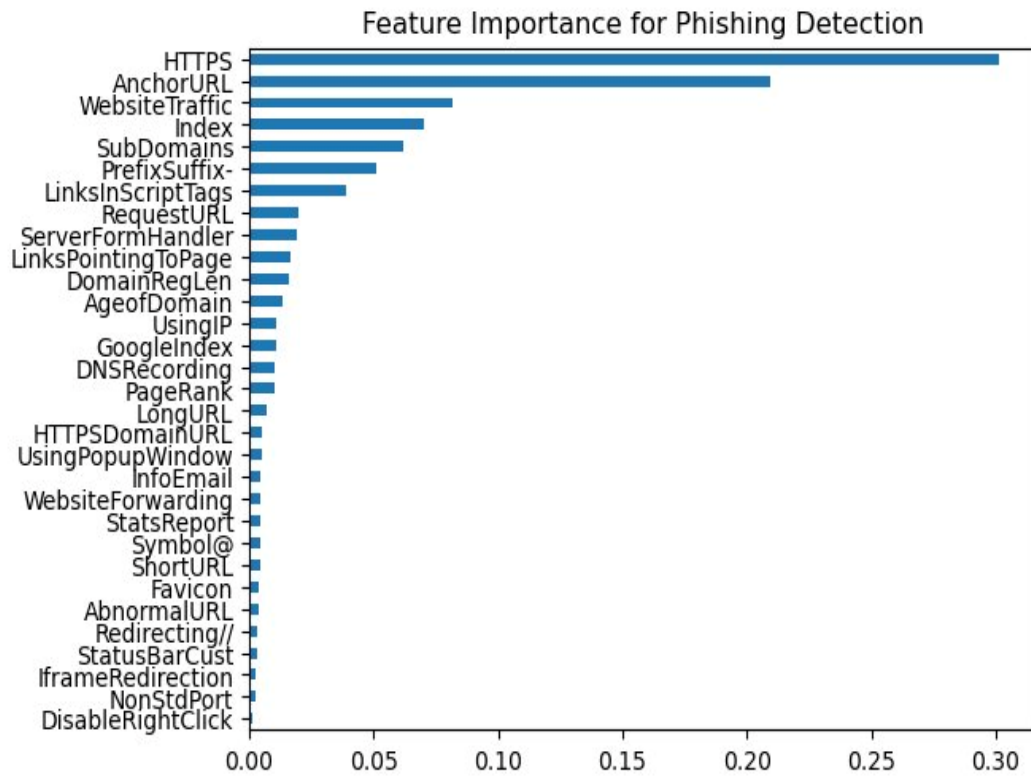


Figure C1: Feature importance scores for all 31 Random Forest features, ranked by mean decrease in Gini impurity. Higher scores indicate greater predictive contribution. Features are those computed from URL structure, HTML content, and network/third-party sources.